Azure Service Bus – Sending a Custom C# Object as a Message

1. What Is the New Concept?

In the previous example, we sent a simple string message:

```
Hello from C# and Visual Studio
```

Now we want to send a **custom C# object**, such as an `Employee` object.

Azure Service Bus messages contain data, so a C# object cannot be sent directly as an object.

We need to:

```
C# Object
    ↓
Serialize
    ↓
JSON String
    ↓
Convert to Bytes
    ↓
ServiceBusMessage
    ↓
Azure Service Bus Queue
```

The main new concepts are:

- **Object Serialization**
- **JSON**
- **Newtonsoft.Json**
- **UTF-8 Encoding**
- Sending serialized object data as the message body

---

2. Why Do We Need Serialization?

Suppose we have this C# object:

```
Employee employee = new Employee
{
    FirstName = "Harsh",
    LastName = "Jane",
    Salary = 50000
};
```

This is an in-memory C# object.

Azure Service Bus does not understand a C# `Employee` object directly.

Therefore, we convert the object into a format that can be transmitted.

A common format is **JSON**.

```
C# Employee Object
        ↓
JSON
```

Example JSON:

```
{
  "FirstName": "Harsh",
  "LastName": "Jane",
  "Salary": 50000
}
```

Now this JSON data can be placed into the Service Bus message.

---

3. What is Serialization?

Serialization means converting an object into a format that can be stored or transmitted.

Example:

```
Employee Object
      ↓
Serialization
```

```
      ↓
JSON
```

Simple Definition

> **Serialization is the process of converting an object into a transferable format such as JSON.**

---

4. What is Deserialization?

The opposite operation is **deserialization**.

It converts the JSON data back into a C# object.

```
JSON
  ↓
Deserialization
  ↓
Employee Object
```

So remember:

```
Object → JSON = Serialization

JSON → Object = Deserialization
```

This becomes especially important when the receiving service needs to reconstruct the original object.

---

5. Employee Class

For this example, assume we have an `Employee` class:

```
public class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public decimal Salary { get; set; }
}
```

Create an object:

```
Employee employee = new Employee
{
    FirstName = "Harsh",
    LastName = "Jane",
    Salary = 50000
};
```

---

6. Install Newtonsoft.Json

To convert the C# object into JSON, the video uses:

```
Newtonsoft.Json
```

Install the NuGet package:

```
Newtonsoft.Json
```

In Visual Studio:

```
Project
   ↓
Manage NuGet Packages
   ↓
Browse
   ↓
Newtonsoft.Json
   ↓
Install
```

---

7. Serialize the Employee Object

Use `JsonConvert.SerializeObject()`:

```
string employeeDataInJson =
    JsonConvert.SerializeObject(employee);
```

This converts:

```
Employee Object
```

into:

```
{
  "FirstName": "Harsh",
  "LastName": "Jane",
  "Salary": 50000
}
```

Now `employeeDataInJson` is a normal C# string.

---

8. Important Method – SerializeObject()

The important method is:

```
JsonConvert.SerializeObject()
```

Example:

```
string json =
    JsonConvert.SerializeObject(employee);
```

Remember

```
JsonConvert.SerializeObject()
                ↓
       Object → JSON
```

---

9. Convert JSON String to Bytes

The `ServiceBusMessage` constructor can accept message body data in byte form.

Therefore, the JSON string is converted into UTF-8 bytes.

The video uses:

```
Encoding.UTF8.GetBytes(employeeDataInJson)
```

Flow:

```
JSON String
    ↓
UTF-8 Encoding
    ↓
Byte Array
```

Example:

```
byte[] employeeBytes =
    Encoding.UTF8.GetBytes(employeeDataInJson);
```

---

10. Why UTF-8?

UTF-8 is a character encoding format used to represent text as bytes.

Since the message is ultimately transferred as data, the JSON string can be converted into bytes using UTF-8.

Simple idea:

```
"Hello"
   ↓
UTF-8
   ↓
Bytes
```

For this example:

```
Employee Object
      ↓
JSON String
      ↓
UTF-8 Bytes
      ↓
ServiceBusMessage
```

---

11. Create ServiceBusMessage

Now create the Service Bus message using the JSON bytes:

```
ServiceBusMessage message =
    new ServiceBusMessage(
        Encoding.UTF8.GetBytes(employeeDataInJson));
```

The message body now contains the serialized employee data.

Conceptually:

```
Employee
   ↓
JSON
   ↓
UTF-8 Bytes
   ↓
ServiceBusMessage
```

---

12. Send the Message

The message can then be sent using the existing sender:

```
await serviceBusSender.SendMessageAsync(message);
```

The new part in this example is not the sending mechanism.

The important change is **how the message body is prepared**.

Previously:

```
String
  ↓
ServiceBusMessage
```

Now:

```
Object
  ↓
Serialize
  ↓
JSON
  ↓
UTF-8 Bytes
  ↓
ServiceBusMessage
```

---

13. Complete Code for This Example

```
using Azure.Messaging.ServiceBus;
using Newtonsoft.Json;
using System.Text;

public class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public decimal Salary { get; set; }
}

string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";
string queueName = "<QUEUE_NAME>";

ServiceBusClient serviceBusClient =
    new ServiceBusClient(connectionString);

ServiceBusSender serviceBusSender =
    serviceBusClient.CreateSender(queueName);

Employee employee = new Employee
{
```

```
    FirstName = "Harsh",
    LastName = "Jane",
    Salary = 50000
};

string employeeDataInJson =
    JsonConvert.SerializeObject(employee);

ServiceBusMessage message =
    new ServiceBusMessage(
        Encoding.UTF8.GetBytes(employeeDataInJson));

await serviceBusSender.SendMessageAsync(message);

Console.WriteLine("Employee message has been sent");
```

---

14. What Actually Happens?

Let's follow the data step-by-step.

Step 1 – Create Object

```
Employee employee = new Employee
{
    FirstName = "Harsh",
    LastName = "Jane",
    Salary = 50000
};
```

Memory contains an `Employee` object.

Step 2 – Serialize

```
JsonConvert.SerializeObject(employee);
```

Result:

```
{
  "FirstName": "Harsh",
  "LastName": "Jane",
```

```
  "Salary": 50000
}
```

Step 3 – Convert to UTF-8 Bytes

```
Encoding.UTF8.GetBytes(employeeDataInJson);
```

Result:

```
JSON String
    ↓
Byte[]
```

Step 4 – Create Message

```
new ServiceBusMessage(bytes);
```

The bytes become the message body.

Step 5 – Send

```
await serviceBusSender.SendMessageAsync(message);
```

The message is sent to the queue.

---

15. Complete Data Flow

```
Employee Object
      |
      | SerializeObject()
      v
JSON String
      |
      | Encoding.UTF8.GetBytes()
      v
```

```
Byte Array
      |
      | ServiceBusMessage
      v
Service Bus Message
      |
      | SendMessageAsync()
      v
Azure Service Bus Queue
```

This is the **most important flow from this video**.

---

16. Verify the Object in Azure Portal

After sending the message, we can verify it using **Service Bus Explorer**.

Go to:

```
Azure Portal
    ↓
Service Bus Namespace
    ↓
Queue
    ↓
Service Bus Explorer
    ↓
Peek
```

The message body should contain the serialized JSON.

Example:

```
{
  "FirstName": "Harsh",
  "LastName": "Jane",
  "Salary": 50000
}
```

This confirms that the C# object was successfully serialized and sent to Azure Service Bus.

---

17. Important Concept – Service Bus Does Not Know About Employee

Azure Service Bus does **not** know that the message originally came from an `Employee` object.

It simply receives message data.

The process is:

```
C# Application
      |
      | Employee Object
      v
Serialization
      |
      | JSON
      v
Service Bus
```

From Service Bus's perspective, it is simply message data.

The receiving application must know how to interpret that JSON.

---

18. Receiving Application

Suppose another service receives this message.

It gets JSON:

```
{
  "FirstName": "Harsh",
  "LastName": "Jane",
  "Salary": 50000
}
```

The receiving application can deserialize it back into an `Employee` object.

Conceptually:

```
Azure Service Bus
      |
      v
JSON
      |
      | Deserialize
      v
Employee Object
```

Example:

```
Employee employee =
    JsonConvert.DeserializeObject<Employee>(json);
```

Therefore, the complete communication becomes:

```
Sender Application
      |
      | Employee Object
      v
Serialization
      |
      | JSON
      v
Azure Service Bus
      |
      | JSON
      v
Receiver Application
      |
      | Deserialization
      v
Employee Object
```

---

19. Serialization vs Deserialization

Serialization	Deserialization
Object → JSON	JSON → Object
Usually performed by sender	Usually performed by receiver

Serialization	Deserialization
Prepares data for transmission	Reconstructs the object
`SerializeObject()`	`DeserializeObject()`

Memory Trick

```
Serialize = Send

Deserialize = Receive
```

More accurately:

```
Object → JSON = Serialize

JSON → Object = Deserialize
```

---

20. Why Do We Use JSON?

JSON is commonly used for communication between applications because it is:

- Human-readable
- Lightweight
- Language-independent
- Easy to serialize and deserialize
- Supported by many programming languages

For example, a .NET application can send JSON and a Java, Python, or Node.js application can consume the same message.

```
.NET
  |
  | JSON
  v
Azure Service Bus
  |
  | JSON
  v
Python / Java / Node.js
```

This makes JSON useful for communication between different technologies.

21. Important Interview Questions

Q1. Can we directly send a C# object to Azure Service Bus?

Answer:

> We normally serialize the C# object into a transferable format such as JSON and then send the serialized data as the Service Bus message body.

Q2. What is serialization?

> Serialization is the process of converting an object into a format such as JSON so that it can be transmitted or stored.

Q3. What is deserialization?

> Deserialization is the process of converting serialized data, such as JSON, back into an object.

Q4. Which library is used in this example for JSON serialization?

```
Newtonsoft.Json
```

Q5. Which method serializes an object?

```
JsonConvert.SerializeObject(object);
```

Q6. Why do we use `Encoding.UTF8.GetBytes()`?

> It converts the JSON string into UTF-8 encoded bytes so that the data can be supplied as the message body.

Q7. What happens to the Employee object before sending?

```
Employee Object
      ↓
JsonConvert.SerializeObject()
      ↓
JSON String
      ↓
Encoding.UTF8.GetBytes()
      ↓
ServiceBusMessage
      ↓
SendMessageAsync()
```

---

22. Quick Revision

The only new flow you need to remember from this video is:

```
C# Object
    ↓
Serialize
    ↓
JSON
    ↓
UTF-8 Bytes
    ↓
ServiceBusMessage
    ↓
Send
```

And on the receiving side:

```
ServiceBusMessage
    ↓
JSON
    ↓
Deserialize
    ↓
C# Object
```

Final Memory Trick

Sender:

> **Object → Serialize → JSON → Bytes → Message → Queue**

Receiver:

> **Message → JSON → Deserialize → Object**

This is the key concept introduced in this video.